

Dokumentation: RNKS Beleg

A.Gröber | s80438

ws23/24

Inhaltsverzeichnis

1	Dokumentation und Lernportfolio (Aufgaben 1-7)	3
1.1	Aufgaben 1 und 2	3
1.2	Vorbereitung auf 3	3
1.3	Aufgabe 3 Erste Schritte	4
1.4	Aufgabe 3/5 Pakete bauen/auf ACKs warten	5
1.5	Änderungen von 1. Abgabeversion zur aktuellen	6
2	Aufgabe 8: Theorieaufgabe	7
3	Aufgabe 9: Funktionalität	9
3.1	grober Programmablauf	9
3.2	Infos zur implementierung	9
3.3	Probleme und Limitierungen	10
3.4	Verbesserungsvorschläge	10
4	Verwendete Editoren,Tutorials,etc.	10
4.1	Editoren	10
4.2	Tutorials Java	10
4.3	Tutorials Latex	10

1 Dokumentation und Lernportfolio (Aufgaben 1-7)

Das folgende Segment dient als Lernportfolio für die Programmieraufgaben 1-7

1.1 Aufgaben 1 und 2

Wie im Screencast gezeigt implementiert. Fenstergröße erstmal auf 1 lassen. Ich versuche erstmal nur stop and wait zu programmieren.

1.2 Vorbereitung auf 3

SW-Diagramme erstellt (im Diagramm Ordner zu finden).

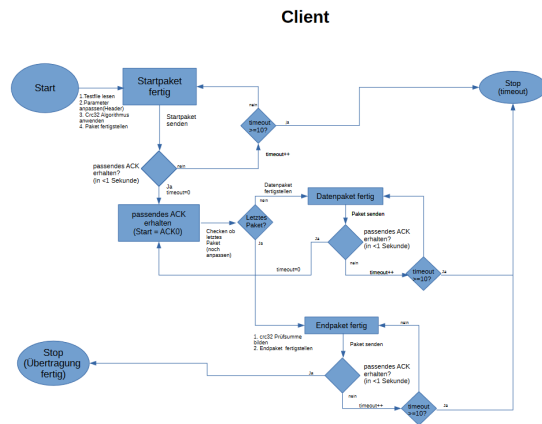


Abbildung 1: Client

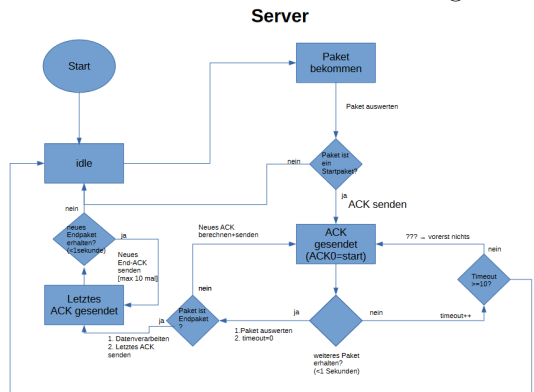


Abbildung 2: Server

1.3 Aufgabe 3 Erste Schritte

Ich habe sehr lange gebraucht um die Klassen und deren Funktionalität etwas zu verstehen, habe es aber geschafft eine Testdatei zu übertragen (ein Datenpaket nach empfangen des ACK0 zu schicken welches selbst auf kein ACK wartet) nächster Schritt ist eine Größere Datei zu verschicken und dieses in einzelne Pakete zu teilen möglicherweise schon ACK's berücksichtigen. Zudem stellt sich die Frage ob einzelne Pakete direkt an die Datei angehängt werden oder erst in einen Puffer oder Tmpdatei (zwischenspeicher) und dann gesammelt in die Datei übertragen werden, was passiert hier wenn Paket fehlerhaft ist?.

Probleme:

ACK wird nicht berücksichtigt

Noch keine Pakete —> nächster Schritte vor Aufgabe 4

Fortschritt:
make.sh+Methode: WriteToFile()

1.4 Aufgabe 3/5 Pakete bauen/auf ACKs warten

Mithilfe von Bytebuffern und deren funktionen [allocate,put*], Startpaket gepackt und abgeschickt. Erstmal in eine Testdatei gespeichert —> alle daten kamen in gewünschter form an. Struktur für Datenpakete + Endpakete erstellt nach selben Schema.

nächster Schritt: Struktur zum Antworten bzw. warten der Acks, (später Timeout Bedingungen und Aufgabe 4)

Fortschritt: Paketstrukturen und erste "Verarbeitung eines Pakets".

Probleme: wenig Verständnis für die Methoden/Funktionen in der Klasse SW

Ich habe ein bisschen mit den Funktionen in SW herumprobiert und erstmal simple Acks (nur ACK:packNr) zurückgegeben. Timeouts noch nicht gesetzt außer in der waitForAck funktion. Startpaket komplett gesendet und empfangen+ausgewertet und die CRC32 summen verglichen. Daraufhin Datei geöffnet und test String darin abgelegt. Dies hat auf localhost ohne Packetverlust funktioniert.

nächster Schritt:

1. Datenpakete auswerten (in angelegte Datei schreiben) und Acks nach Protokoll verschicken+verarbeiten +Endpacket identifizieren und verarbeiten
2. Timeouts setzen
3. wenn möglich den Server/Port "reseten", sodass mehrere clients nacheinander dateiene senden können.
4. auf Kanal mit delay und fehler rate testen.

Fortschritt: Vollständige Struktur beim Sender, beim Server fehlt die verarbeitung des Endpakets und rückgabe vollständiger Acks

Acks wie im Protokoll vorgesehen implementiert außer für letztes Paket.

Problem: wie unterscheidet Server letztes Paket von einen normalen daten Paket, provisorische Lösung End-PacketNr=127 senden.

Auf localhost große Datei verschickt diese mittels crc32-summen gecheckt, diese wird jetzt Fehlerlos in das Verzeichnis Upload/ kopiert. Allerdings funktionieren nur Dateiformate die keine Nullbytes enthalten (Text funktioniert, png nicht), Da Ich mittels der Funktion filterNullbytes überschüssige Nullbytes von der Request entferne (falls das Dateiformat diese enthält, wird das Paket nur bis zur Stelle des ersten Nullbytes nach der Paketnr eingelesen)

Workarounds:Paketnr letztes Paket=127 | nur bestimmte Formate erlaubt

nächster Schritt: EndAck implementieren, Timeouts einfügen, Workarounds bearbeiten wenn mögl.

Fortschritt: Datei senden ohne fehler möglich, crc32 summen stimmen für startpaket sowie für ganze datei überein. Server kann mehrere Dateien hinteinander empfangen.

Keine Lösung für das Problem mit dem letzten Paket zur Unterscheidung vom normalen Daten-Paket, daher ist das letzte Paket mit PaketNr 127 gekennzeichnet (sowie das entsprechende Ack=Fack). Ganze Datei wird jetzt übertragen und Fack kommt wie gewünscht an mit richtigem CRC32 Wert.
nächster Schritt: Timeouts -> auf channel mit Fehlern testen

Timeouts fertig, auf localhost ist eine Datei Übertragung mit fehlerbehaftetem Kanalmöglich (Testszenario: 10KB=10DP+1SP+1FP mit bis zu 0.5 Fehlerrate und 200ms delay)
nächster Schritt: Aufgabe 4

test logger auskommentiert, Datenraten mittels System.currentTimeMillis() und Datasize bzw. Filesize implementiert
nächster Schritt: Mit VPN auf Hochschulservern testen.
1. Abgabe hier

1.5 Änderungen von 1. Abgabeverision zur aktuellen

Client: letztes Datenpaket von Endpaket getrennt (Endpaket jetzt nur 7Byte)
Das hat auch auf dem Hochschulserver funktioniert

Server: Eigentliches Paket (max 1027Byte) aus dem vom Socket gesendeten Paket (65555Byte [null]) extrahieren. Neue Struktur zum verarbeiten

Jedes Dateiformat und beliebig viele Datenpakete können gesendet werden.
Lokal sowie auf Hochschulserver.

Server weiter angepasst und 60 Sekunden Timeout hinzugefügt, CloseConnection eingefügt.

1 Workaround/Probleme:

1. Um normales Datenpaket von letztem Datenpaket voneinander zu unterscheiden, wird die Größe des Pakets mit einer Variable verglichen. Die globale Variable datasize-check gibt die zu vergleichende Größe für das 1.DP an und sollte so groß sein wie Datasize (Datenanteil im Datenpaket vom Client). Das ist unsauber gelöst und funktioniert nur wenn Datasize >= datasize-check.

2 Aufgabe 8: Theorieaufgabe

Ethernetgeschwindigkeit-link

Grundlage der Rechnung im Bsp der Sicherungsschicht abgehandelt.

$L=1400\text{Byte}+3\text{Byte}$ (Seasionnummer+Paketnummer) gerundet 1400 Start+Endpaket
sind kleiner daher vernachlässigbar in der Rechnung

geg.:

$$L = 1.400 \text{ Byte} = 11.200 \text{ Bit}$$

$$T_w = 0,01 \text{ s}$$

$$P_e = P_r = 0,1$$

$$R = 1$$

ges.:

$$\eta_{sw} = \frac{T_p}{T_p + T_w} (1 - P_e)(1 - P_r) * R$$

lsg.:

$$T_p = \frac{L}{r_b} = \frac{11.200 \text{ Bit}}{94/2 \text{ MBit/s}} = 0.00024 \text{ s}$$

ohne fehler:

$$\eta_{sw} = \frac{0,00024 \text{ s}}{0,00024 \text{ s} + 0.01 \text{ s}}$$

$$\eta_{sw} = 0.023$$

Symbolfehlerrate mit eigentlichen Datendurchsatz multiplizieren

$$0.023 * 94/2 \text{ MBit/s} = 1.081 \text{ MBit/s}$$

$$1.081 \text{ MBit/s} * 1000 = 1081 \text{ Kbit/s}$$

$$930,6 \text{ Kbit/s} \div 8 = 135.125 \text{ KB/s}$$

mit fehler

$$\eta_{sw} = \frac{0,00024 \text{ s}}{0,00024 \text{ s} + 0.01 \text{ s}} * 0.9 * 0.9 * 1$$

$$\eta_{sw} = 0.018$$

Symbolfehlerrate mit eigentlichen Datendurchsatz multiplizieren

$$0,0198 * 94/2 \text{ MBit/s} = 0.9306 \text{ MBit/s}$$

$$0.9306 \text{ MBit/s} * 1000 = 930,6 \text{ Kbit/s}$$

$$930,6 \text{ Kbit/s} \div 8 = 116,325 \text{ KB/s}$$

Info: Ich habe ein Kbyte als 1000 byte (bzw. 1Mbyte als 1000Kbyte) angenommen, in der Realität sind es 1024! Das vereinfacht die Rechnung und ist nicht wirklich ausschlaggebend auf das Ergebnis.

Warum ist Praktische Anwendung (Beleg) langsamer?

Auf meinem Localhost komme ich mit genannten Kanalparametern auf eine Datenrate von etwa 30-60 Kbit/s. (das stimmt etwa mit dem Hochschulserver überein) Das lässt sich zum einen durch die relativ langen Timeouts erklären zum anderen ist meine Packetgröße nur 1024 Byte was einen maximalen Datendurchsatz von etwa 80 KB/s zulässt. Am stärksten fallen allerdings die Timeouts ins Gewicht.

Bearbeitungs bzw. Berechnungszeit der Pakete fallen aus dem Gewicht da diese sehr klein sind. Ausgenommen von den Crc32-checks im StartPaket und letzten Ack.

3 Aufgabe 9: Funktionalität

3.1 grober Programmablauf

Server und Client können separat gestartet werden. Im Fall das der Server/Port nicht bereitsteht wird der Client nach 10 Sekunden geschlossen.

Der Server wartet bis zu 5 Minuten auf ein Paket bevor er die Verbindung schließt. Es ist möglich mehrere Client nacheinander auf dem selben Port zu bedienen.

Der Client verschickt einzelne Pakete, diese werden in der Klasse Filetransfer erstellt und mittels der Funktion myARQ.data-req in die Klasse SW übergeben. Wo Sie zum Teil ausgelesen werden, um zu bestimmen welches Paket verschickt wird.

Innerhalb von data-req (SW-Klasse) wird das Paket über den Socket verschickt und auf eine Antwort (ACK) gewartet.

Auf Server Seite wird der Socket permanent abgehört bis ein Paket eintrifft, falls es ein Startpaket ist wird eine Session geöffnet (konkret wird eine Datei mit dem übergebenen Dateinamen und der Dateigröße geöffnet und ein paar globale Variablen gesetzt).

Im Fall eines Datenpakets werden die Daten des Pakets hinten an die Datei angehängt, selbiges gilt für das letzte Datenpaket.

Falls ein Ack oder Paket verloren geht, wird es bis zu 10 mal erneut gesendet.

Um ein neues Ack zu erzeugen (für ein bereits bekanntes Paket), checkt die Funktion SW.data-ind-req ob Sie bereits ein Startpaket erhalten hat bzw. die Paketnummer. Im Fall, dass das Paket bereits bearbeitet wurde sendet die Funktion nur ein ÄckString an den Server.

Die Verbindung zum Client und der Clint an sich, wird sobald das letzte Ack (FAck) eingetroffen ist, geschlossen. Es wird auf der Konsole ausgegeben ob beide CRC32-Summen gleich sind

3.2 Infos zur implementierung

1. Ich habe die Größe des Datenteils beim Datenpaket auf 1024bytes=1Kbyte festgelegt. Diese kann mit der Variable Datasize angepasst werden.
2. Bei der Konvertierung der CRC32-Summe (long zu int) im Startpaket wird die Summe negativ, das verursacht hier erstmal keine Probleme sollte aber möglicherweise angepasst werden. [Idee hierfür: int mit max value initialisieren und dann die CRC32 summe davon subtrahieren].
3. Pakete sind so implementiert wie es da Protokoll vorsieht.
4. Es ist nur das Stop and Wait Protokoll implementiert.

3.3 Probleme und Limitierungen

Timeouts sind immer 1 Sekunde was sich auf den Datendurchsatz auswirkt (im Fall das Pakete verloren gehen). Eine bessere Strategie wäre vielleicht den ersten Timeout kurz zu wählen (timeout 1: 200ms), und mit voranschreitenden Timouts, längere Timeout zeiten zu wählen (Timeout 2: 300ms usw.)

3.4 Verbesserungsvorschläge

- dynamische Timeouts (Protokoll bzw.Aufgabenstellung)
- positive CRC32-Summe im Header (meine Implementation)
- andere Lösung für letztes Datenpaket (meine Implementation)
- Verbesserte Serverstruktur (meine Implementation)

4 Verwendete Editoren,Tutorials,etc.

4.1 Editoren

JDK: openjdk 11.0.21 2023-10-17
IntelliJ IDEA (Programmieren+debugen)
VS-Code (Programmieren+Latexdokumentation)
libreoffice draw (erstellen der Diagramme)

4.2 Tutorials Java

w3 type-casting + andere Tutorials
overflow bytearray kurz
overflow java int zu long
crc32-java
crc-allg
unsigned int java
aufrunden genutzt in Datenpaket berechnung

4.3 Tutorials Latex

<https://www.overleaf.com/learn/latex/Hyperlinks>
https://en.wikibooks.org/wiki/LaTeX/Title_Creation
<https://www.namsu.de/Extra/pakete/Graphicx.html>
Youtube-Playlist hauptsächlich Part3